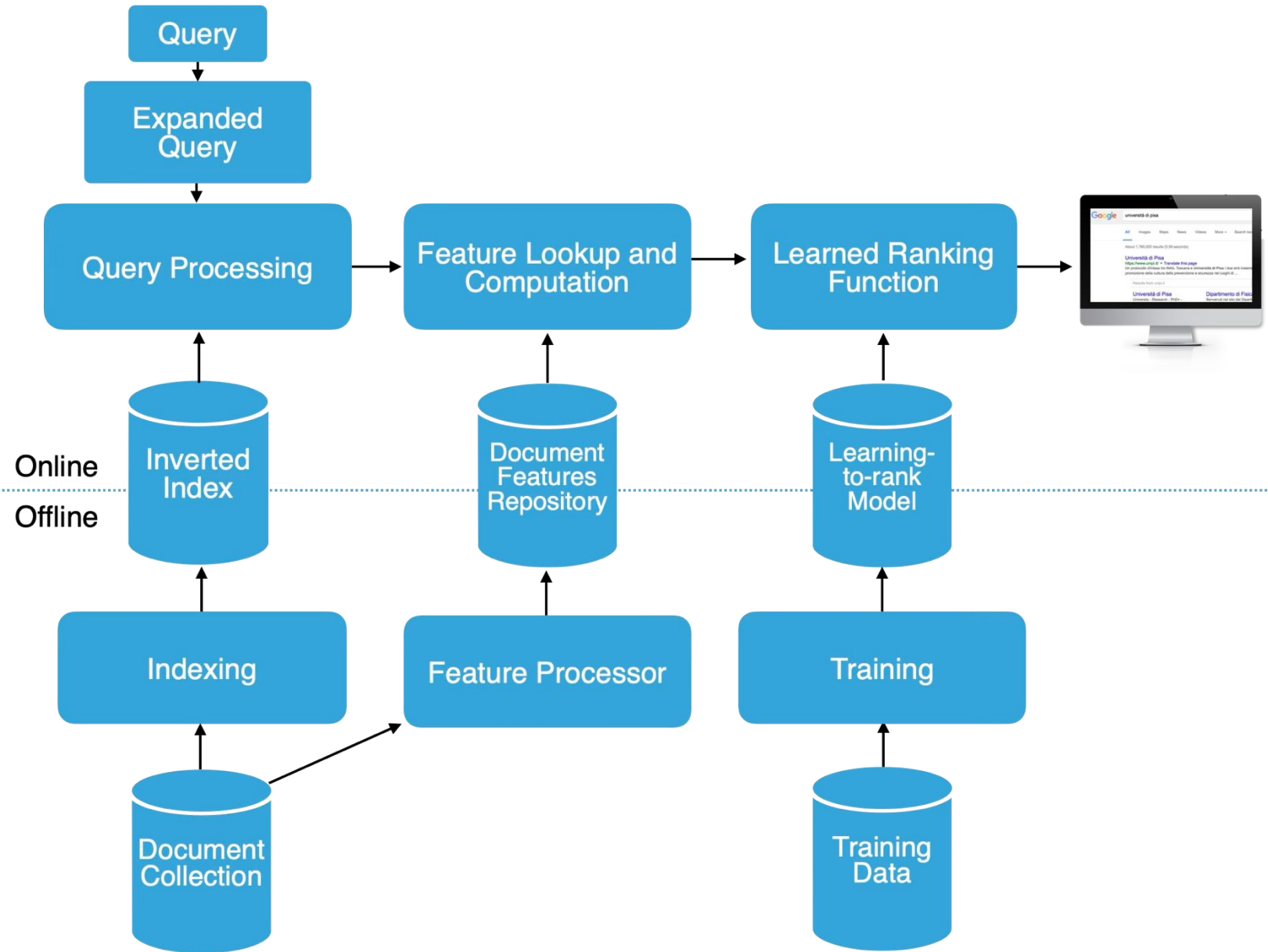


SCORING, TERM WEIGHTING AND THE VECTOR SPACE MODEL



Ranked retrieval

- Thus far, our queries have all been Boolean.
 - Documents either match or don't.
- Good for expert users with precise understanding of their needs and the collection.
 - Also good for applications: Applications can easily consume 1000s of results.
- Not good for most users.
 - Incapable of writing Boolean queries (or they are, but they think it's too much work).
 - Don't want to wade through 1000s of results.
 - This is particularly true of web search.

Problem with Boolean search: feast or famine

- Boolean queries often result in either too few (=0) or too many (1000s) results.
- It takes a lot of skill to come up with a query that produces a manageable number of hits.
 - AND gives too few; OR gives too many

Ranked retrieval models

- Rather than a set of documents satisfying a query expression, in **ranked retrieval**, the system reorders the (top) documents in the collection for a given query
- **Free text queries**: Rather than a query language of operators and expressions, the user's query is just one or more words in a natural language
 - the user specifies his information need through an **ad-hoc query** which initiates a search
- **Ranked retrieval** has normally been associated with free text queries and vice versa

Feast or famine: not a problem in ranked retrieval

- When a system produces a ranked result set, large result sets are not an issue
 - Indeed, the size of the result set is not an issue
 - We just show the top k (≈ 10) results
 - We don't overwhelm the user
- Premise: the ranking algorithm works 😊

Scoring as the basis of ranked retrieval

- We wish to return “in order” those documents
 - First the ones most likely to be useful to users
- How can we rank-order the documents in the collection with respect to a query?
 - Assign a **score** – say in $[0, 1]$ – to each document given a query
 - This score measures how well document and query “match”
- We need a way of assigning a score to a query/document pair

Jaccard coefficient for scoring?

- Recall: A commonly used measure of overlap of two sets A and B
- $\text{jaccard}(A, B) = |A \cap B| / |A \cup B|$
- $\text{jaccard}(A, A) = 1$
- $\text{jaccard}(A, B) = 0$ if $A \cap B = 0$
- A and B don't have to be of the same size.
- Always assigns a number between 0 and 1.

Jaccard coefficient for scoring

- What is the query-document Jaccard score for each of the two documents below?
 - Query: *ides of march*
 - Document 1: *caesar died in march*
 - Document 2: *the long march*
- $J(q, d1) = 1 / (3+4-1) = 1/6$
- $J(q, d2) = 1 / (3+3-1) = 1/5$
- Using Jaccard $d2$ is more similar to q than $d1$ ☹️
 - However, the issue here is not Jaccard

Issues with Jaccard for scoring

- It doesn't consider *term frequency* (how many times a term occurs in a document)
 - Jaccard doesn't consider this information, as texts are represented as “sets” of terms, as in the Boolean model of IR
- Term frequency
 - The **more frequent** the query term in the document, the **higher the score** (should be)
 - ***Rare terms*** in a collection are more informative than ***frequent terms***

Binary term-document incidence matrix

lec 6.2

10/11

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	1	1	0	0	0	1
Brutus	1	1	0	1	0	0
Caesar	1	1	0	1	1	1
Calpurnia	0	1	0	0	0	0
Cleopatra	1	0	0	0	0	0
mercy	1	0	1	1	1	1
worser	1	0	1	1	1	0

Each document is represented by a binary vector $\in \{0,1\}^{|V|}$

Term-document **count matrices**

- Consider the **number of occurrences** of a term in a document:
 - Each document is a **count vector** in $\mathbb{N}^v$: a column below
 - Each count vector is a bag of words (a multiset)

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	157	73	0	0	0	0
Brutus	4	157	0	1	0	0
Caesar	232	227	0	2	1	1
Calpurnia	0	10	0	0	0	0
Cleopatra	57	0	0	0	0	0
mercy	2	0	3	5	5	1
worser	2	0	1	1	1	0

Bag of words model (multiset)

- Count vector (bag of words) representation doesn't consider the ordering of words in a document
 - *John is quicker than Mary* and *Mary is quicker than John* have the same vectors

Term frequency tf

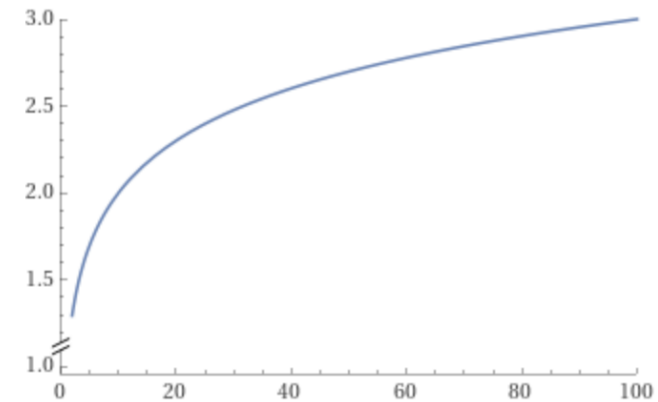
- The term frequency $\mathbf{tf}_{t,d}$ of term t in document d is defined as the **count/number of times** that t occurs in d
- We want to use $\mathbf{tf}_{t,d}$ when computing query-document match scores. But how?
- Raw term frequency is not what we want:
 - A document with 10 occurrences of the term is more relevant than a document with 1 occurrence of the term.
 - But not 10 times more relevant
- Relevance does not increase proportionally with term frequency

NB: frequency = count in IR

Log-frequency weighting

- Log-frequency weight of term t in d to diminish its effects:

$$w_{t,d} = \begin{cases} 1 + \log_{10} \text{tf}_{t,d}, & \text{if } \text{tf}_{t,d} > 0 \\ 0, & \text{otherwise} \end{cases}$$



- $0 \rightarrow 0, 1 \rightarrow 1, 2 \rightarrow 1.3, 10 \rightarrow 2, 1000 \rightarrow 4$, etc.
- **Score** for a document-query pair: sum over terms t in both q and d :

$$\text{Score}_{qd} = \sum_{t \in q \cap d} (1 + \log \text{tf}_{t,d})$$

- The score is 0 for document d if none of the query terms q is present in d .

Document frequency

- **Rare terms** are more informative than frequent terms
- Consider a term in the query that is (globally) rare in the collection (e.g., *arachnocentric*)
 - A document containing this term is very likely to be relevant to the query *arachnocentric*
 - We want higher weights for rare terms like *arachnocentric*

Document frequency, continued

- Frequent terms are less informative than rare terms
 - Consider a query term that is frequent in the collection (e.g., *high*, *increase*, *line*)
 - A document containing such a term is more likely to be relevant than a document that doesn't
 - But it's **not a sure indicator** of relevance wrt the user information need, as a **large part of the documents** contains this query term

Document frequency, continued

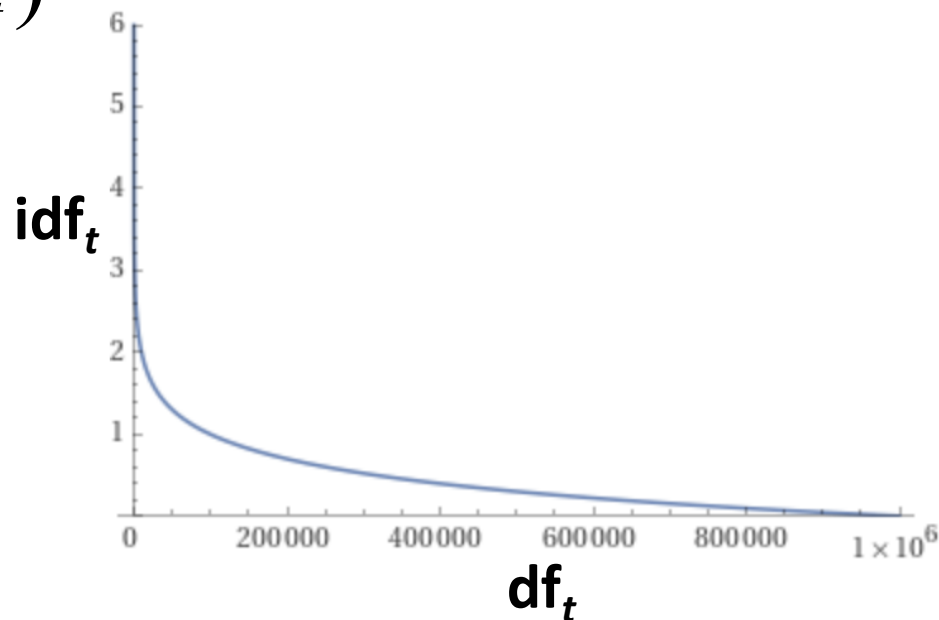
- We will use document frequency (df_t) to capture this
 - we want **high positive weights** for words like *high*, *increase*, and *line* (characterized by a large df_t , but much smaller than the collection size)
 - but **lower** than **weights** assigned to **globally rare terms**, characterized by a very small df_t

idf weight

- df_t is the document frequency of t : the number of documents that contain t
 - df_t is an **inverse measure** of the **informativeness** of t
 - $df_t \leq N$, where N is the no. of documents in the collection
- We define the idf (*inverse document frequency*) of t

by $idf_t = \log_{10} (N/df_t)$

- We use **$\log (N/df_t)$** instead of **N/df_t** to “dampen / decrease” the effect of df_t



idf example, suppose $N = 1$ million

term	df_t	idf_t
calpurnia	1	6
animal	100	4
sunday	1,000	3
fly	10,000	2
under	100,000	1
the	1,000,000	0

$$idf_t = \log_{10} (N/df_t)$$

There is the same idf value for each term t in the collection, and it can be precomputed as it's query-independent

Effect of *idf* on ranking

- Does *idf* have an effect on ranking for one-term queries, like “iPhone”?
- *idf* doesn't rank documents for "one term" queries
 - all documents in which the term occurs have the same score:

$$\text{Score}_{qd} = \sum_{t \in q \cap d} \log_{10} \left(\frac{N}{df_t} \right)$$

- *idf* alone can produce a ranking of the matching documents only for queries with at least two terms (if all docs do not contain all query terms)
 - For the query **capricious person**, *idf* weighting makes occurrences of **capricious** count for much more in the final document ranking than occurrences of **person**.

Collection vs. Document frequency

- The *collection frequency* $\mathbf{cf}_t$ of $\mathbf{t}$ is the number of occurrences of $\mathbf{t}$ in the collection (see Zipf's law), thus counting multiple occurrences in each document.
- Example:

Word	Collection frequency	Document frequency
<i>Insurance</i>	10440	3997
<i>Try</i>	10422	8760

- Which word is a better search term (and should get a higher weight)?

How do we weight doc terms in the vectors?

- Here's the intuition:

- Terms that appear often in a document (**tf**) should get high weights

The more often a document contains the term “dog”, the more likely that the document is “about” dogs.

- Terms that appear in many documents (**df**) should get low weights

Words like “the”, “a”, “of” appear in (nearly) all documents.

- How do we capture this mathematically?

- Term frequency
- Inverse document frequency

tf-idf weighting



- The tf-idf weight of a term is the product of its tf weight (normalized *wf*) and its idf weight.

$$\text{idf}_t = \log \frac{N}{\text{df}_t} \quad \text{wf}_{t,d} = \begin{cases} 1 + \log \text{tf}_{t,d} & \text{if } \text{tf}_{t,d} > 0 \\ 0 & \text{otherwise} \end{cases}$$

$$\text{wf-idf}_{t,d} = \text{wf}_{t,d} \times \text{idf}_t$$

Alternative smaller:
 $\log(1 + \text{tf}_{t,d})$

- **Best known weighting scheme in Information Retrieval**
 - Note: the “-” in tf-idf is a hyphen, not a minus sign!
 - **Alternative names: tf.idf, tf x idf**
- Increases with the number of occurrences within a document
- Increases with the rarity of the term in the collection

Score for a document given a query

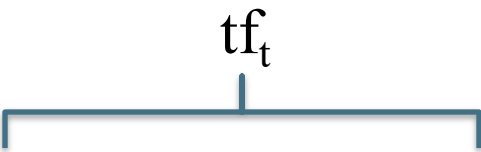
$$\text{Score}(q, d) = \sum_{t \in q \cap d} \text{tf.idf}_{t,d}$$

- There are many variants
 - How “tf” is computed (with/without logs, discretizing tf values)
 - Whether the terms in the query are also weighted
 - ...

Example

$$\text{idf}_t = \log \frac{N}{\text{df}_t}$$

$N = 806,791$ docs



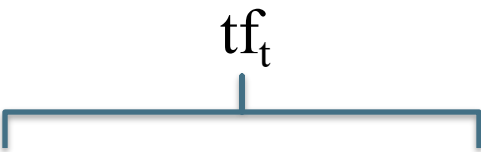
	Doc1	Doc2	Doc3	df _t	idf _t
car	27	4	24	18,165	1.65
auto	3	33	0	6723	2.08
insurance	0	33	29	19,241	1.62
best	14	0	17	25,235	1.5

- Compute the tf-idf weights for the query terms **car**, **auto**, **insurance**, **best** for each document, using the *idf* in the collection: $\mathbf{tf}_{t,d} \times \mathbf{idf}_t$

Example

$$\text{idf}_t = \log \frac{N}{\text{df}_t}$$

$N = 806,791$ docs



	Doc1	Doc2	Doc3	df_t	idf_t
car	27	4	24	18,165	1.65
auto	3	33	0	6723	2.08
insurance	0	33	29	19,241	1.62
best	14	0	17	25,235	1.5

- Compute the tf-idf weights for the query terms **car**, **auto**, **insurance**, **best** for each document, using the *idf* in the collection: $\text{tf}_{t,d} \times \text{idf}_t$

- Doc1

- car: $27 \times 1.65 = 44.55$

- auto: $3 \times 2.08 = 6.24$

- insurance: 0

- best: $14 \times 1.5 = 21$

- Using $\log(1 + \text{tf}_{t,d})$, idf_t becomes more important

- car: $1.45 \times 1.65 = 2.39$

- auto: $0.6 \times 2.08 = 1.2543$

- insurance: 0

- best: $1.18 \times 1.5 = 1.764$

Binary $\rightarrow$ count $\rightarrow$ weight matrix

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	5.25	3.18	0	0	0	0.35
Brutus	1.21	6.1	0	1	0	0
Caesar	8.59	2.54	0	1.51	0.25	0
Calpurnia	0	1.54	0	0	0	0
Cleopatra	2.85	0	0	0	0	0
mercy	1.51	0	1.9	0.12	5.25	0.88
worser	1.37	0	0.11	4.15	0.25	1.95

Each document is now represented by a real-valued vector of tf-idf weights $\in \mathbf{R}^{|V|}$

where V is the vocabulary of the collection

Documents as vectors

- So, we have a $|\mathbf{V}|$ -dimensional vector space
- Vocabulary's terms are axes of the space
- Documents are points or vectors in this space
- Very high-dimensional: tens of millions of dimensions when you apply this to a web search engine
- Very sparse vectors - most entries are zero.

Queries as vectors

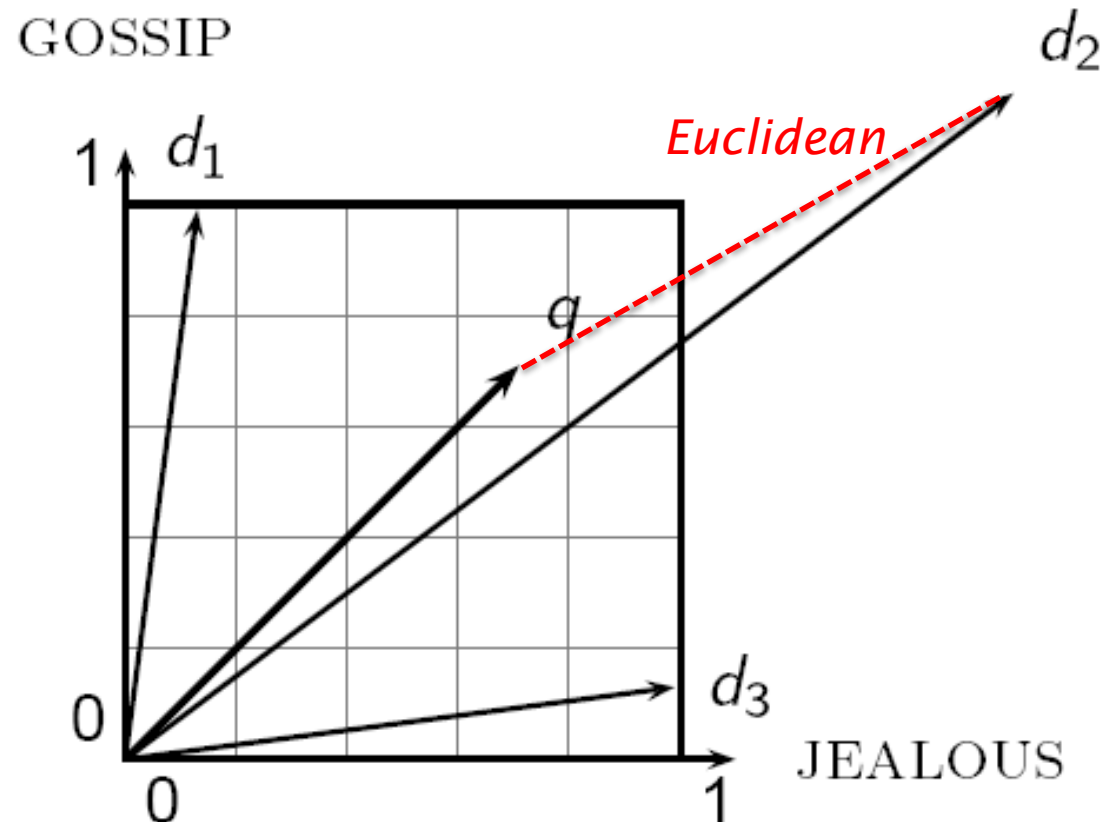
- Key idea 1: Do the same for queries: represent them as vectors in the space
- Key idea 2: Rank documents according to their proximity to the query in this space
 - proximity = similarity of vectors
 - proximity $\approx$ inverse of distance
- Recall: We do this because we want to get away from the either-in-or-out of the Boolean model.
- Instead: rank more relevant documents higher than less relevant documents

Formalizing vector space proximity

- First cut: distance between two points
 - (= distance between the end points of the two vectors)
- **Euclidean distance?**
- Euclidean distance is a bad idea . . .
- . . . because Euclidean distance is **large** for vectors of **different lengths**.
- Euclidean distance works however for normalized vectors

Why distance is a bad idea

The Euclidean distance between $\vec{q}$ and $\vec{d}_2$ is large even though the distribution of terms in the query $\vec{q}$ and the distribution of terms in the document $\vec{d}_2$ are very similar.

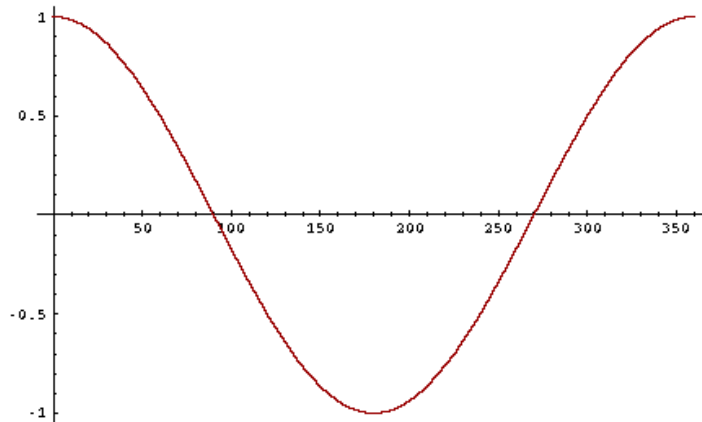


Use angle instead of distance

- Experiment: take a document d and append it to itself. Call this document d'
 - “Semantically” d' and d have the same content
 - Vectors d' and d overlap, but the magnitudes d' is double
- The Euclidean distance between the two documents can be quite large
 - Although the **angle** between the two vectorial representations of the two documents is 0° (in the vector space)
- Key idea: Rank document vectors according to the “angle” with the query vector.

From angles to cosines

- The following two notions are equivalent.
 - Rank documents in increasing order of the angle between query and document ← **angle=distance**
 - Rank documents in decreasing order of cosine(query,document) ← **cosine=similarity**
- Cosine is a monotonically decreasing function for the interval $[0^\circ, 180^\circ]$



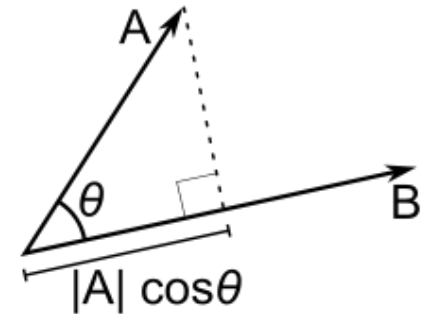
Recall: dot-product

- Algebraic definition

$$\mathbf{A} \cdot \mathbf{B} = \sum_{i=1}^n A_i B_i = A_1 B_1 + A_2 B_2 + \cdots + A_n B_n$$

- Geometric definition

$$\mathbf{A} \cdot \mathbf{B} = \|\mathbf{A}\| \|\mathbf{B}\| \cos \theta,$$



where the magnitude of $\mathbf{A}$ (Euclidean length) is

L_2 norm $\|\mathbf{A}\| = \sqrt{\mathbf{A} \cdot \mathbf{A}},$

and thus:

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \cdot \|\mathbf{B}\|} = \frac{\mathbf{A}}{\|\mathbf{A}\|} \cdot \frac{\mathbf{B}}{\|\mathbf{B}\|}$$

Vector length normalization

Length normalization

- A vector can be (length-) normalized by dividing each of its components by its length – for this we use the L_2 norm:

$$\|\vec{x}\|_2 = \sqrt{\sum_i x_i^2}$$

- Dividing a vector by its L_2 norm makes it a **unit (length) vector** (on the surface of the **unit hypersphere**)
- Effect on the two documents d and d' (d appended to itself) from the earlier slide: they have identical vectors after length-normalization
 - Long and short documents have now comparable weights

cosine(query,document)

$$\cos(\vec{q}, \vec{d}) = \frac{\vec{q} \cdot \vec{d}}{|\vec{q}| |\vec{d}|} = \frac{\vec{q}}{|\vec{q}|} \cdot \frac{\vec{d}}{|\vec{d}|} = \frac{\sum_{i=1}^{|V|} q_i d_i}{\sqrt{\sum_{i=1}^{|V|} q_i^2} \sqrt{\sum_{i=1}^{|V|} d_i^2}}$$

Dot product
Unit vectors

q_i is the tf-idf weight of term i in the query

d_i is the tf-idf weight of term i in the document

$\cos(\vec{q}, \vec{d})$ is the cosine similarity of $\vec{q}$ and $\vec{d}$... or,
equivalently, the cosine of the angle between $\vec{q}$ and $\vec{d}$.

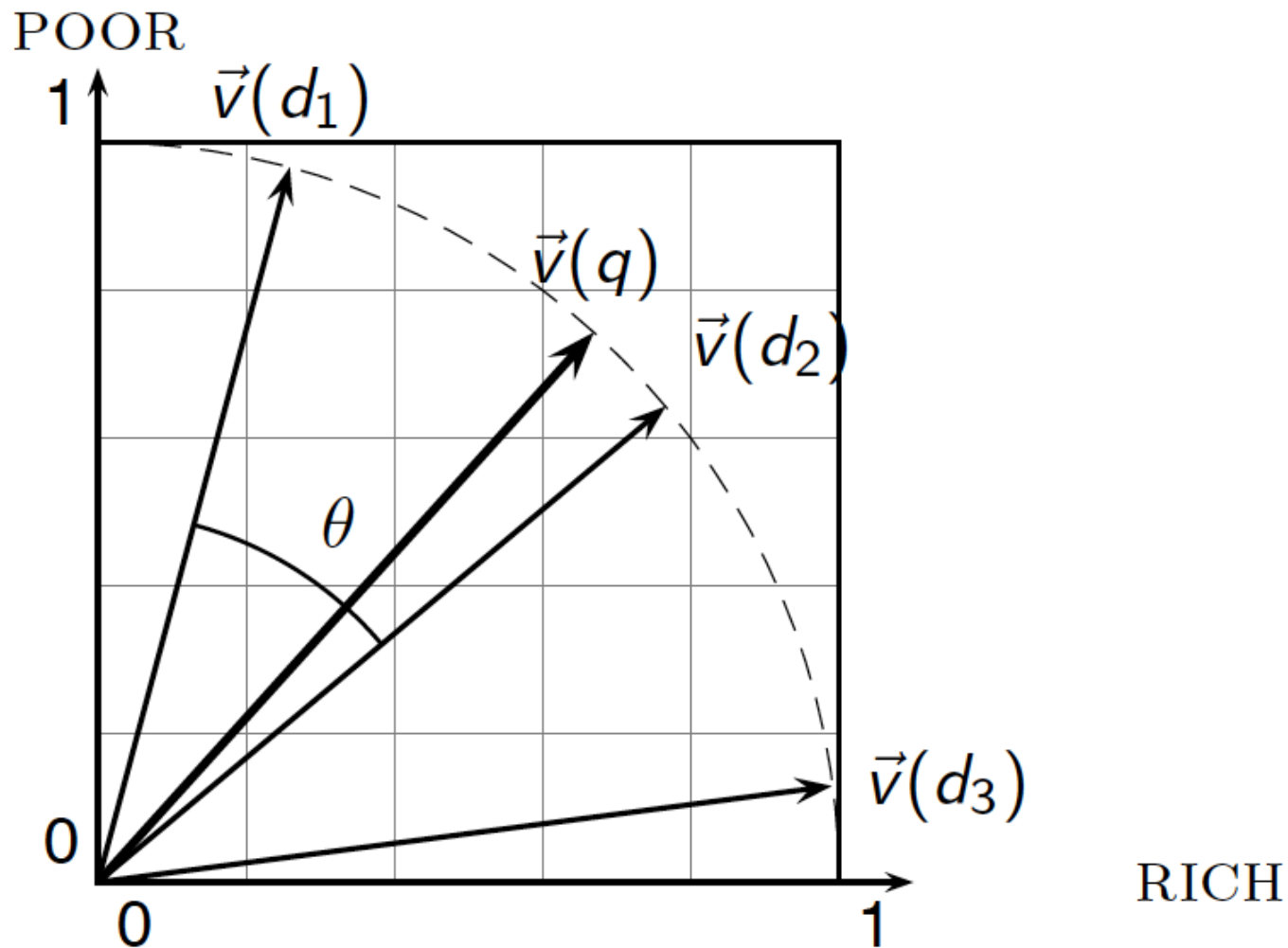
Cosine for length-normalized vectors

- For length-normalized vectors, cosine similarity is simply the dot product (or scalar product):

$$\cos(\vec{q}, \vec{d}) = \vec{q} \cdot \vec{d} = \sum_{i=1}^{|V|} q_i d_i$$

for q, d length-normalized.

Cosine similarity illustrated



Cosine similarity amongst 3 documents

e.g., for document clustering purpose

- How similar are the novels
 - **SaS**: *Sense and Sensibility*
 - **PaP**: *Pride and Prejudice*
 - **WH**: *Wuthering Heights*?

term	SaS	PaP	WH
affection	115	58	20
jealous	10	7	11
gossip	2	0	6
wuthering	0	0	38

Term frequencies (counts)



Jane Austen (1775 –1817)

SaS & PaP



Emily J. Brontë (1818 –1848)

WH

3 documents example contd.

Log frequency weighting

term	SaS	PaP	WH
affection	3.06	2.76	2.30
jealous	2.00	1.85	2.04
gossip	1.30	0	1.78
wuthering	0	0	2.58

After length normalization

term	SaS	PaP	WH
affection	0.789	0.832	0.524
jealous	0.515	0.555	0.465
gossip	0.335	0	0.405
wuthering	0	0	0.588

$$\cos(\text{SaS}, \text{PaP}) \approx$$

$$0.789 \times 0.832 + 0.515 \times 0.555 + 0.335 \times 0.0 + 0.0 \times 0.0 \approx \mathbf{0.94}$$

$$\cos(\text{SaS}, \text{WH}) \approx \mathbf{0.79}$$

$$\cos(\text{PaP}, \text{WH}) \approx \mathbf{0.69}$$

$$\cos(\text{SaS}, \text{PaP}) > \cos(\text{SaS}, \text{WH}) > \cos(\text{PaP}, \text{WH})$$

tf-idf weighting has many variants

Term frequency		Document frequency		Normalization	
n (natural)	$tf_{t,d}$	n (no)	1	n (none)	1
l (logarithm)	$1 + \log(tf_{t,d})$	t (idf)	$\log \frac{N}{df_t}$	c (cosine)	$\frac{1}{\sqrt{w_1^2 + w_2^2 + \dots + w_M^2}}$
a (augmented)	$0.5 + \frac{0.5 \times tf_{t,d}}{\max_t(tf_{t,d})}$	p (prob idf)	$\max\{0, \log \frac{N - df_t}{df_t}\}$	u (pivoted unique)	$1/u$
b (boolean)	$\begin{cases} 1 & \text{if } tf_{t,d} > 0 \\ 0 & \text{otherwise} \end{cases}$			b (byte size)	$1/CharLength^\alpha$, $\alpha < 1$
L (log ave)	$\frac{1 + \log(tf_{t,d})}{1 + \log(\text{ave}_{t \in d}(tf_{t,d}))}$				

Weighting may differ in queries vs. documents

- Many search engines allow for different weightings for queries vs. documents
- **SMART Notation:** denotes the combination in use in an engine, with the notation *ddd.qqq*, using the acronyms from the previous table
- A very standard weighting scheme is: **Inc.ltc**
- **Document:** logarithmic tf (l as first character), **no idf** (n in the 2nd column), cosine normalization
- **Query:** logarithmic tf (l in leftmost column), **idf** (t in 2nd column), cosine normalization ...

tf-idf example: Inc.Itc

idf
only in the query

Document: *car insurance auto insurance*

Query: *best car insurance*

Term	Query (Itc)						Document (Inc)				Prod
	tf-raw	tf-wt	df	idf	wt	n' lize	tf-raw	tf-wt	wt	n' lize	
auto	0	0	5000	2.3	0	0	1	1	1	0.52	0
best	1	1	50000	1.3	1.3	0.34	0	0	0	0	0
car	1	1	10000	2.0	2.0	0.52	1	1	1	0.52	0.27
insurance	1	1	1000	3.0	3.0	0.78	2	1.3	1.3	0.68	0.53

Exercise: what is N , the number of docs for idf computation?

$$\text{Doc length} = \sqrt{1^2 + 0^2 + 1^2 + 1.3^2} \approx 1.92$$

$$\text{Score} = 0 + 0 + 0.27 + 0.53 = 0.8$$

Summary – vector space ranking

- Represent the query as a weighted tf-idf vector
- Represent each document as a weighted tf-idf vector
- Compute the cosine similarity score for the query vector and each document vector
- Rank documents with respect to the query by score
- Return the top K (e.g., $K = 10$) to the user

Best Match 25

- ... also known as BM25, full name “Okapi BM25”
 - developed under the *Okapi information retrieval system*, implemented at London's City University in the 1980s and 1990s.
- used everywhere nowadays (strong baseline)



Centre for Interactive Systems Research

The OKAPI Information Retrieval System

Okapi is

.... an experimental information retrieval system, developed in the [Centre for Interactive Systems Research](#) in the [Department of Information Science](#) at [City University](#), London.

Okapi represents

.... twelve years of research in [probabilistic retrieval](#) and information-seeking behaviour.

Okapi functions as

.... a platform for testing and evaluation, via "live" and "laboratory" experiments.

BM25

$$\text{score}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)}$$

$$\text{IDF}(q_i) = \ln\left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1\right)$$

- $|D|$: length of the document in # words
- avgdl: average length of the documents in the collection
- b and k_1 are hyper-parameters (should be tuned on the collection)
 - b in $[0,1]$ (0.75 typically), k_1 in $[1.2, 2.0]$

BM25

$IDF_t = \log(N/n_t)$
is the inverse
document frequency

$$BM25(d, q) = \sum_t IDF_t \tau(F_t)$$

$$F_t = \frac{f_{t,d}}{1 - b + b \cdot l_d / L}$$

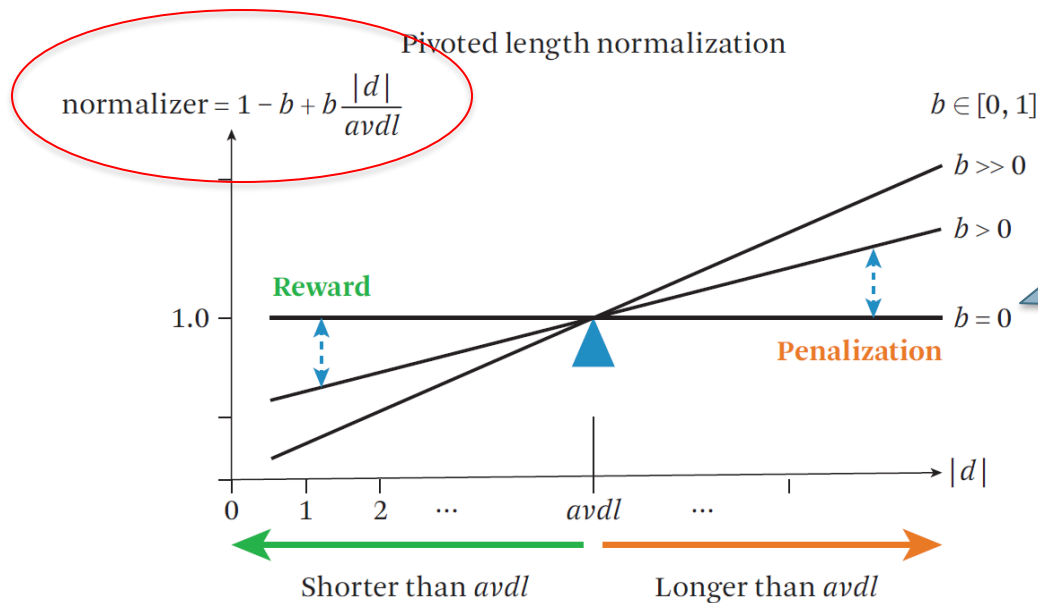
If $l_d = L$, then
 $F_t = TF$

- $f_{t,d}$ is the frequency of term t in document d
- l_d is the length of document d
 - longer documents are less important
- L is the average document length in the collection
- b determines the importance of l_d

BM25

Pivoted transformation for Doc length,
where $L = avdl$ (average doc length)

$$F_t = \frac{f_{t,d}}{1 - b + b \cdot l_d / L}$$



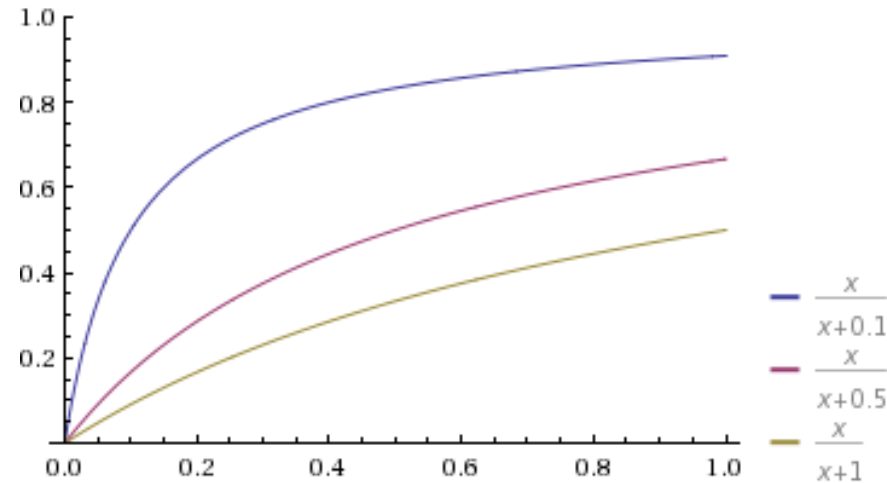
For $b > 0$, the normalizer (denominator) becomes larger than 1.0

We aim to penalize long documents because they naturally have a better chance to match any query.

BM25

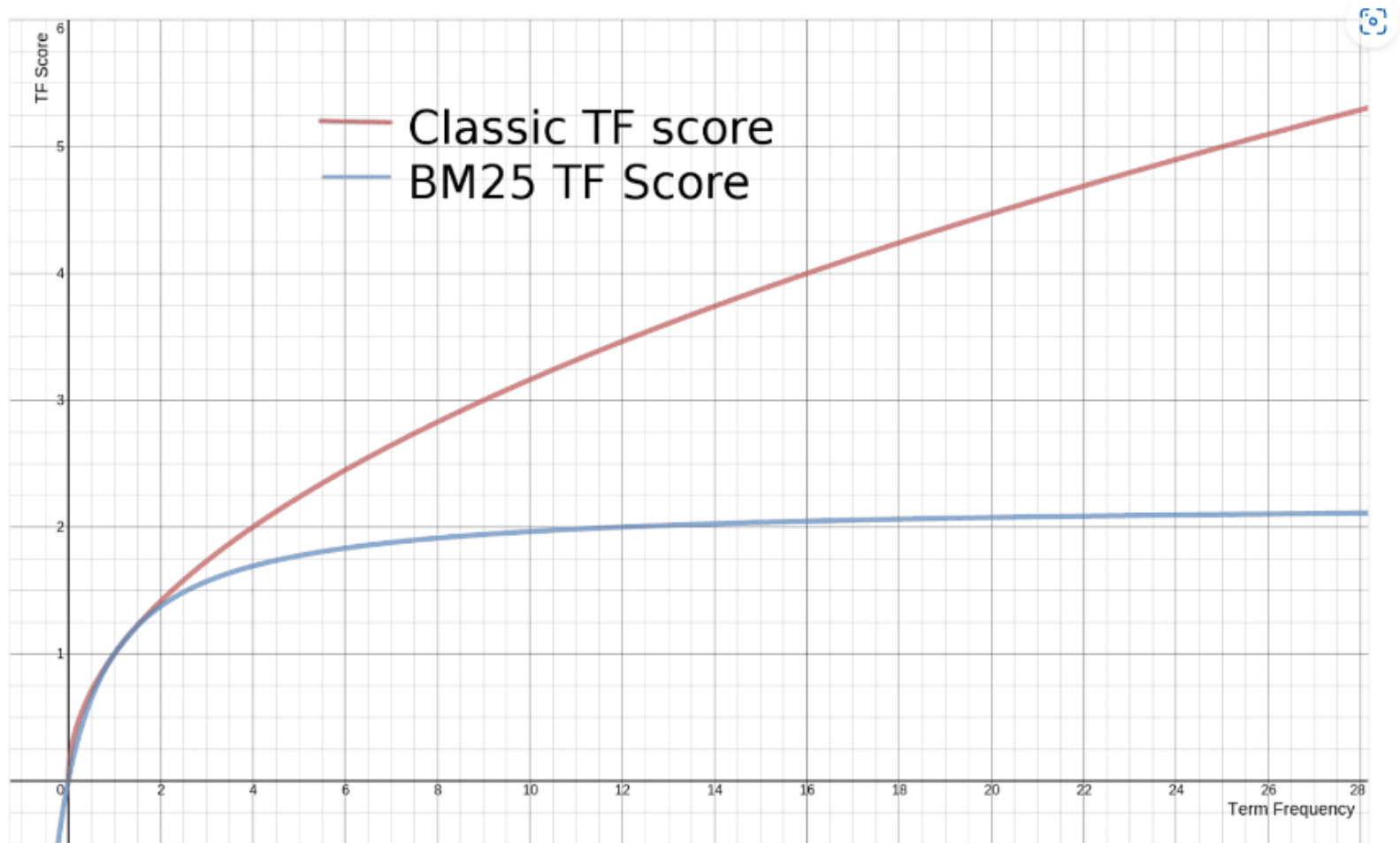
$$\text{BM25}(d, q) = \sum_t \text{IDF}_t \tau(F_t)$$

$$\tau(F_t) = \frac{F_t}{k + F_t}$$



- τ is a **saturation function**, modeling *non-linearity* of term frequencies contribution, in turn normalized with document length
- Without tuning the two hyperparameters, k between 1.2 and 2, and $b = 0.75$.

BM25



Resources for today's lecture

- IIR 6.2 – 6.4.3
- Stephen Robertson and Hugo Zaragoza. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. Found. Trends Inf. Retr. 3, 4 (April 2009), 333–389. <https://doi.org/10.1561/15000000019>